

OPENREELpy - FUNCTION GUIDE

Page 1 - Architecture

OPENREELpy is a FastAPI ASGI application exported as app from app.py. Search validation, dork construction, Archive queries, metadata selection and playable-file URL generation run in Python. Bootstrap 5, Tailwind CSS and Font Awesome support the interface. Minimal browser JavaScript renders API responses, controls dialogs and drives native video.

The API routes are /api/health, /api/search and /api/media/{identifier}. FastAPI also serves the HTML shell, allowlisted documents and a mounted static directory. There is no database, secret or background worker.

Page 2 - Search Compiler and Safety

Python `validate_query()` validates length and checks piracy-related wording. `build_dork()` removes quote and angle-bracket characters, normalises whitespace and generates an archive-restricted query using MP4, WebM and OGV alternatives plus open-rights phrases.

Python `build_archive_query()` tokenises the request. `/api/search` uses async httpx, a twelve-second timeout, repeated field parameters, download-count sorting and movie media type.

The dork and Archive query are related but distinct. The user sees and edits the dork; the structured API supplies the in-app result index. This avoids scraping Google and keeps result fields machine-readable.

Page 3 - Metadata and Playback

`/api/media/{identifier}` validates identifiers against a strict allowlist and requests Archive metadata. Python `playable_files()` accepts non-private MP4, WebM or OGV files and recognised video formats. Thumbnail, sample and trailer filenames are excluded. The largest eligible file becomes the primary stream.

The source URL is assembled from encoded identifier and path segments. `mime()` supplies an appropriate video MIME type. Native video uses `preload=metadata` and `playsinline`; media bytes are not downloaded until the browser needs them.

Supplementary controls call standard media methods and properties. Fullscreen uses optional chaining because support varies. Error handling inserts a readable failure notice without concealing the source link.

Page 4 - Storage and Rendering

openreelpy-watchlist and openreelpy-history contain JSON arrays capped at sixty entries. Browser preferences remain client-side because Vercel Functions have an ephemeral filesystem and must not be used for durable local-file persistence.

All dynamic values pass through `esc()` before HTML insertion. The query is URL encoded before external navigation. Thumbnail and source paths encode identifiers. External links use protective relationship attributes.

Documentation is fetched from `/docs`. FastAPI's allowlisted route blocks traversal and arbitrary-file reads. Markdown is escaped and displayed as text. CONTACT is the only iframe and uses a sandbox plus strict referrer policy.

Page 5 - Extension and Security Boundaries

New providers should be limited to lawful, documented public APIs with clear rights metadata and CORS support. Do not add general web scraping, torrent indexes, DRM circumvention or proxying. Preserve the source-record link and make provider failure explicit.

Adaptive streaming would require a reviewed HLS or DASH library and careful Content Security Policy changes. Caption support should prefer provider-supplied WebVTT tracks with validated URLs. Recommendations should remain transparent and local unless users explicitly consent to a remote profile.

Vercel runs the app as one Function with Fluid compute by default. Keep calls bounded, never proxy media, and keep the bundle small. Static mounts are supported and may be CDN-promoted. After changes, run syntax, compiler, safety, metadata, modal, accessibility, responsive, document-pagination and packaging tests.